Project 3 Writeup

Cricket Bergner

04/01/26

1 Introduction

Project 3 applies numeric integration, differentiation, and ODE solving techniques to a simple harmonic oscillator (SHO) system. Three different integration methods were tested, scipy.odeint, RK45, and Velocity-Verlet. These methods had varying degrees of accuracy when it came to mapping the SHO, which will be explored through this writeup.

1.1 Languages and Libraries

The code for this project was written in Python. The libraries used were matplotlib.pyplot, numpy, scipy.integrate, and scipy.spatial. The former two were used mainly for math functions and plotting. Scipy.integrate had some interesting and unique additions that contributed heavily to the code development.

Scipy.integrate had two imported sub-modules, odeint and solve_ivp. Odeint inputs the function, initial x or y values, the time, and a derivative function that is defined by the user. With these it is able to solve the ODE on its own, without additional input from the user. Similarly, solve_ivp takes in similar inputs (albeit in a different order) and also solves the ODE on its own. For this code it was used in regards to the Velocity-Verlet integrator. Instead of passing in a derivative function, an acceleration function was passed in instead, which was alternate version of the derivative function.

Scipy.spatial contributed an important function called ConvexHull. This function was able to take in points, locate vertices, and find the area or volume of the shape the vertices created. In the case of this project, it was able to map how the area of a mesh grid of points shifted over time under certain initial conditions.

2 Undergirding Theory

2.1 RK45

The Runge-Kutta-Fehlberg method, also known as RK45, is an algorithm for the numerical solution of ordinary differential equations. This method produces a fourth order solution with a fifth order error estimate. The difference between the fourth and fifth order approximations

gives an estimate of the local error. This allows for the method to have an adaptive step size, taking smaller steps when the function is changing rapidly and larger steps when the function is not changing much.

2.2 Velocity-Verlet

With this method, the velocity and position are calculated simultaneously. Here, at each time step, half-step updates for position and velocity are calculated using the standard kinematic equations. It is worth noting that this algorithm assumes acceleration only depends on position.

$$v\left(t+\frac{1}{2}\Delta t\right) = v(t) + \frac{1}{2}a(t)\Delta t \tag{1}$$

$$x(t+\Delta t) = x(t) + v\left(t+\frac{1}{2}\Delta t\right)\Delta t \tag{2}$$

$$a(t+\Delta t) = a(x(t+\Delta t)) \tag{3}$$

$$v(t+\Delta t) = v\left(t+\frac{1}{2}\Delta t\right) + \frac{1}{2}a(t+\Delta t)\Delta t \tag{4}$$

$$\tag{5}$$

Since the midpoint method is employed, this method of integration has an error of $O(h^2)$.

2.3 The Simple Harmonic Oscillator

A simple harmonic oscillator (SHO) is a system that has been displaced from its equilibrium position, but a restoring force proportional to the displacement helps drive it back to its equilibrium position. Think of a mass on a spring. The system is modeled by a second order differential equation, where k is the spring constant and m is the mass.

$$m\ddot{x} + kx = 0 \tag{6}$$

If a force proportional to the velocity is acting on the mass (think of friction), then the system is thought of as a damped SHO, where b is the damping coefficient.

$$m\ddot{x} + b\dot{x} + kx = 0 \tag{7}$$

3 Walking Through the Code

Before the code, all the relevant libraries are imported. All functions from the header file are imported as well. Those functions will be discussed below when they are called in the main body. They were moved to a header file for better visual clarity in the code.

Listing 1: Importing the relevant libraries.

```
# import libraries
import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import odeint, solve_ivp
from scipy.spatial import ConvexHull
from P3_header import *
```

3.1 The Three Symplectic Integrators

All of the constants are defined. These will help specify the wave function so the integrators can approximate an exact numerical solution. There are three symplectic integrators.

The first is referred to as "scipy" throughout the code. This integration method uses the odeint function, which solves and returns values for each step in the time array. This method switches between taking tiny steps over rapidly changing curves and larger steps over smooth curves to robustly calculate a solution. This integration method will call on a derivative function that is called "deriv_odeint".

Listing 2: Scipy symplectic integrator and associated derivative calculator for the odeint function.

```
# derivative function for the scipy integrator
def deriv_odeint(N, t, omega, damping):
    x, v = N
    dxdt = v
    dvdt = -omega**2*x - damping*v
    return [dxdt, dvdt]

# Scipy Method
def scipy_method(N_initial, tmin, tmax, nts, deriv):
    t = np.linspace(tmin, tmax, nts)
    N = odeint(deriv, N_initial, t)
    x = N[:,0]
    v = N[:,1]

    return t, x, v
```

The second integrator is called "RK45", as it is the Runge-Kutta-Fehlberg method. Scipy has a function that uses this method, called "solve_ivp." The code and associated derivative function do not differ significantly from the "odeint" method, other than how the inputs are passed in.

Listing 3: Scipy RK45 symplectic integrator and associated derivative calculator for the solve_ivp function.

```
# derivative function for the RK45 symplectic integrator
def deriv_ivp(t, N, omega, damping):
```

```
    x, v = N
    dxdt = v
    dvdt = -omega**2*x - damping*v
    return [dxdt, dvdt]

# RK4(5) scipy function
def RK45_method(N_initial, tmin, tmax, nts, deriv):
  t = np.linspace(tmin, tmax, nts)
  sol = solve_ivp(deriv, (tmin, tmax), N_initial, method='RK45', t_eval=t)
  x = sol.y[0]
  v = sol.y[1]

  return t, x, v
```

The third integrator is called "Velocity-Verlet". This method takes in a time range and a number of steps, then calculates the position using velocity and acceleration. The velocity is updated using an average of acceleration and time. Then, a loop will jump to the next step for the process to begin again. This integrator uses yet another derivative approximation function, this one dubbed "accel" as acceleration is the main derivative used in the execution of the Verlet method.

Listing 4: Velocity-Verlet symplectic integrator and its associated acceleration function.

```
def accel(x, v, omega, damping):
    return -omega**2*x - damping*v

# velocity-verlet symplectic integrator
def verlet(N_initial, tmin, tmax, nts, deriv):

  # define inital variables
  x0, v0 = N_initial
  x = np.zeros(nts + 1)
  v = np.zeros(nts + 1)
  x[0] = x0
  v[0] = v0
  t = np.linspace(tmin, tmax, nts + 1)
  dt = t[1] - t[0]

  # loop
  for i in range(nts):
    a = deriv(x[i], v[i])
    v_half = v[i] + 0.5 * dt * a
    x[i+1] = x[i] + dt * v_half
    a_new = deriv(x[i+1], v_half)
    v[i+1] = v_half + 0.5 * dt * a_new

  return t, x, v
```

3.2 The Phase Space of a SHO

The three integrators are called to give arrays for time, position, and velocity. Since the SHO is not damped, the damping variable passed into the functions is set to zero. Plotting phase space is plotting position versus velocity. All three integrators are plotted on the same graph and given different line thicknesses to show them apart. The errors were also calculated. These were computed by using the Verlet integrator as a baseline and calculating how the Scipy and RK45 functions drifted from that.

Listing 5: How the phase space plots are constructed for the undamped SHO, and how error is calculated.

```
# call the integrators
t_s, x_s, v_s = scipy_method(N_initial, tmin, tmax, nts,
    lambda N, t_odeint: deriv_odeint(N, t_odeint, omega, damping))
t_r, x_r, v_r = RK45_method(N_initial, tmin, tmax, nts,
    lambda t_ivp, N_ivp: deriv_ivp(t_ivp, N_ivp, omega, damping))
t_v, x_v, v_v = verlet(N_initial, tmin, tmax, nts,
    lambda x, v: accel(x, v, omega, damping))

# make the plots
plt.plot(v_r, x_r, color = '#f37fb0', label = "RK45", linewidth = 10)
plt.plot(v_v, x_v, 'o', color = '#61dbb0', label = "Verlet")
plt.plot(v_s, x_s, color = '#6316b0', label = "Scipy")
plt.xlabel("Velocity (m/s)")
plt.ylabel("Position (m)")
plt.title("Phase Space of a Harmonic Oscillator")
plt.legend()
plt.show()
print("")

# compute errors relative to Verlet
error_x_r = (x_r[:nts] - x_v[:nts])
error_v_r = (v_r[:nts] - v_v[:nts])

error_x_v = (x_v[:nts] - x_v[:nts])
error_v_v = (v_v[:nts] - v_v[:nts])

error_x_s = (x_s[:nts] - x_v[:nts])
error_v_s = (v_s[:nts] - v_v[:nts])

error_r = np.sqrt(error_x_r**2 + error_v_r**2)
error_s = np.sqrt(error_x_s**2 + error_v_s**2)
error_v = np.sqrt(error_x_v**2 + error_v_v**2)
```

The error arrays used a similar code to produce a log-log plot. Both plots are shown below.

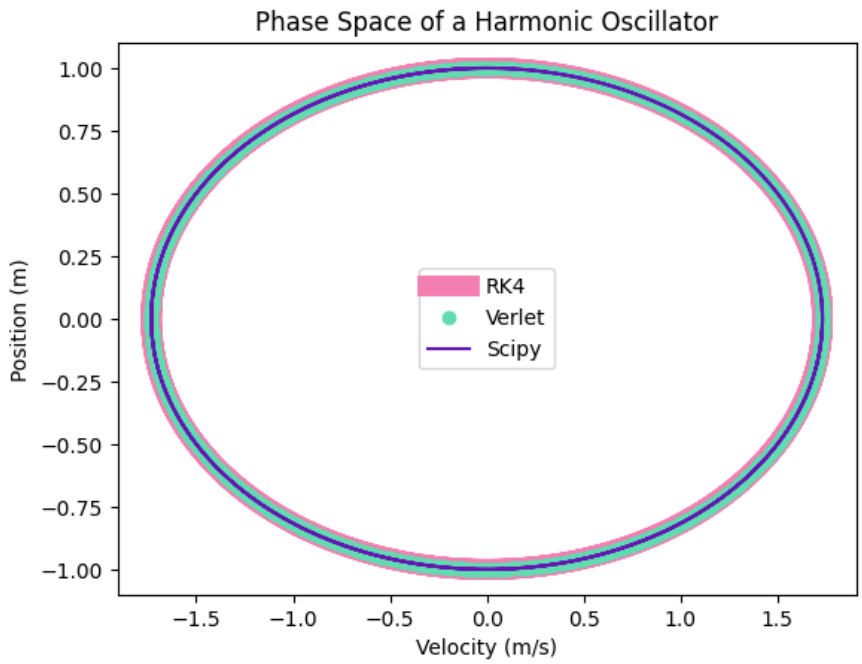

Figure 1: The phase space of a SHO as calculated by the three integrators.

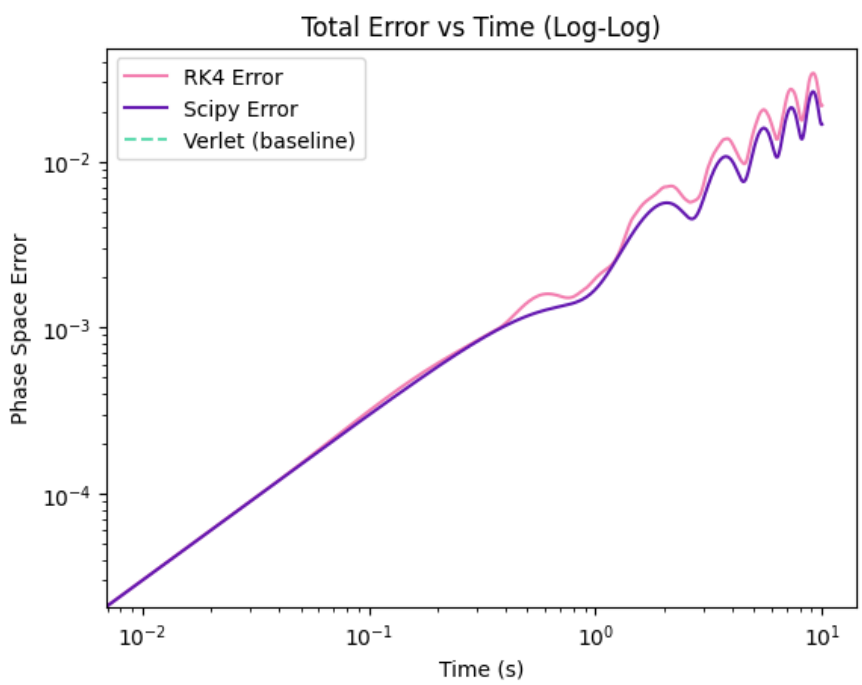

Figure 2: The error of the Scipy and RK45 integrators as compared to the Verlet integrator in calculating the phase space of an undamped SHO.

The code for the damped SHO did not vary much from the undamped case. The damping variable was given a value of 0.2. Once again, the three integrators were called to produce

position, velocity, and time arrays, in which the position was plotted against the velocity to generate the phase space. The error was calculated in the same way as before.

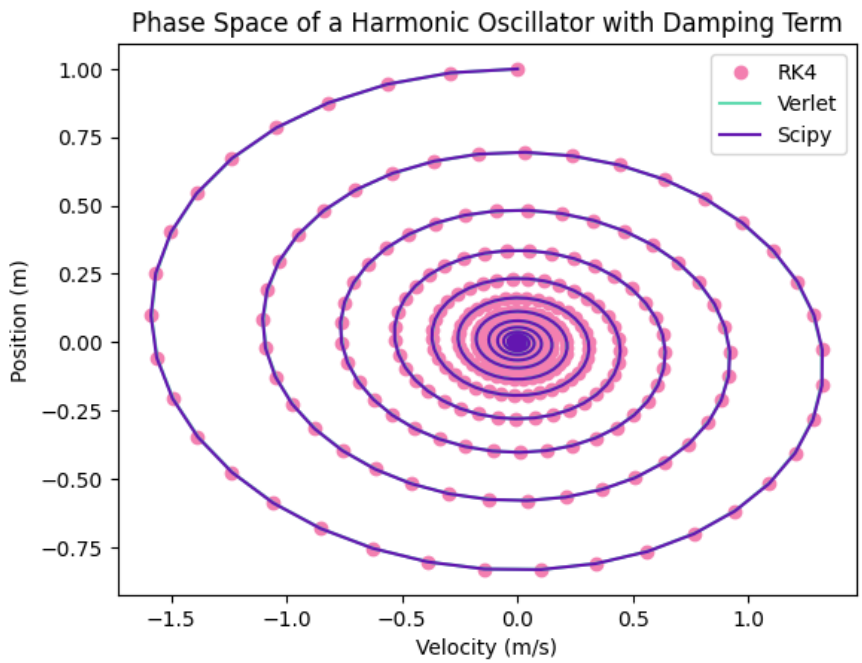

Figure 3: The phase space of a damped SHO as calculated by the three integrators.

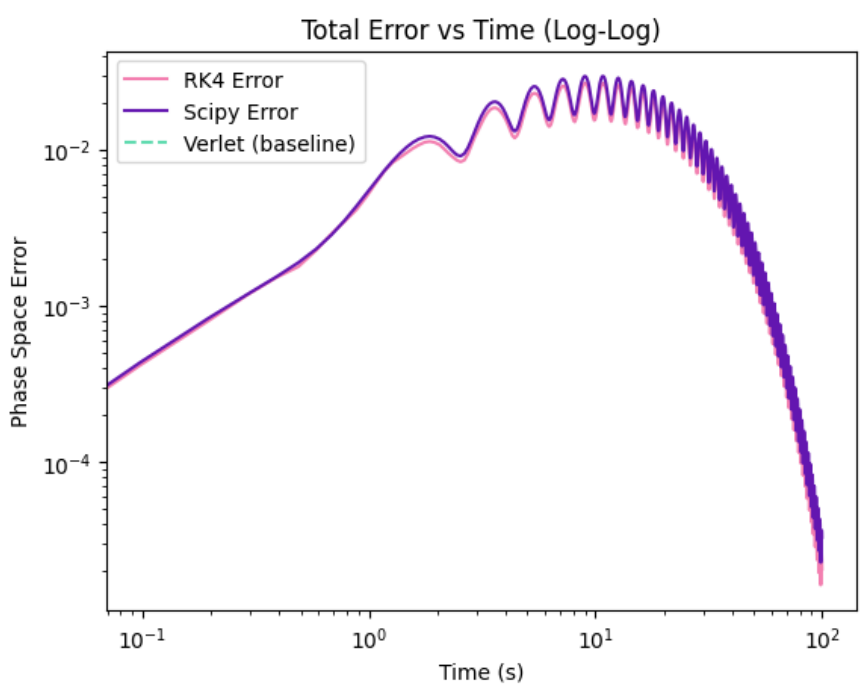

Figure 4: The error of the Scipy and RK45 integrators as compared to the Verlet integrator in calculating the phase space of a damped SHO.

3.3 Total Mechanical Energy of a SHO

The total mechanical energy was found via $E = K + U = (1/2)mv^2 + (1/2)kx^2$. This equation was applied to all three integrators, then the energy vs time plot was generated.

Listing 6: Calculating total mechanical energy from all three integrators and plotting the energy versus time.

```
# total mechanical energy
E_r = 0.5*m*v_r**2 + 0.5*k*x_r**2
E_v = 0.5*m*v_v**2 + 0.5*k*x_v**2
E_s = 0.5*m*v_s**2 + 0.5*k*x_s**2

# make the plots
plt.plot(t_r, E_r, color = '#f37fb0', label = "RK4")
plt.plot(t_v[:nts], E_v[:nts], color = '#61dbb0', label = "Verlet")
plt.plot(t_s, E_s, color = '#6316b0', label = "Scipy")
plt.xlabel("Time (s)")
plt.ylabel("Energy (J)")
plt.title("Total Mechanical Energy Over Time")
plt.legend()
plt.show()
print("")
```

The errors were computed in a similar manner to the phase space error, where the energy for the Verlet method was used as a baseline and the other two integrators were measured against it.

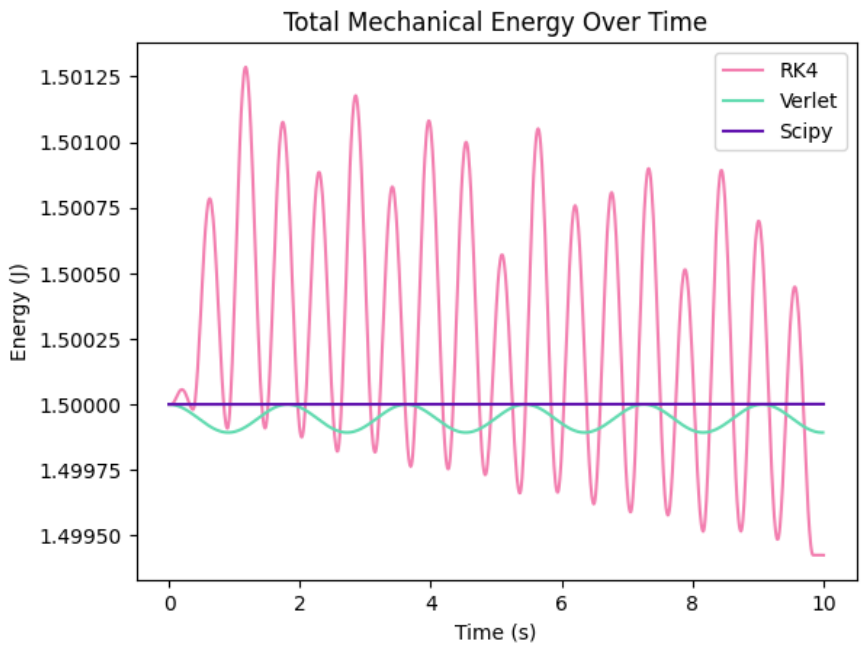

Figure 5: The total mechanical energy versus time of an undamped SHO as calculated by the three integrators.

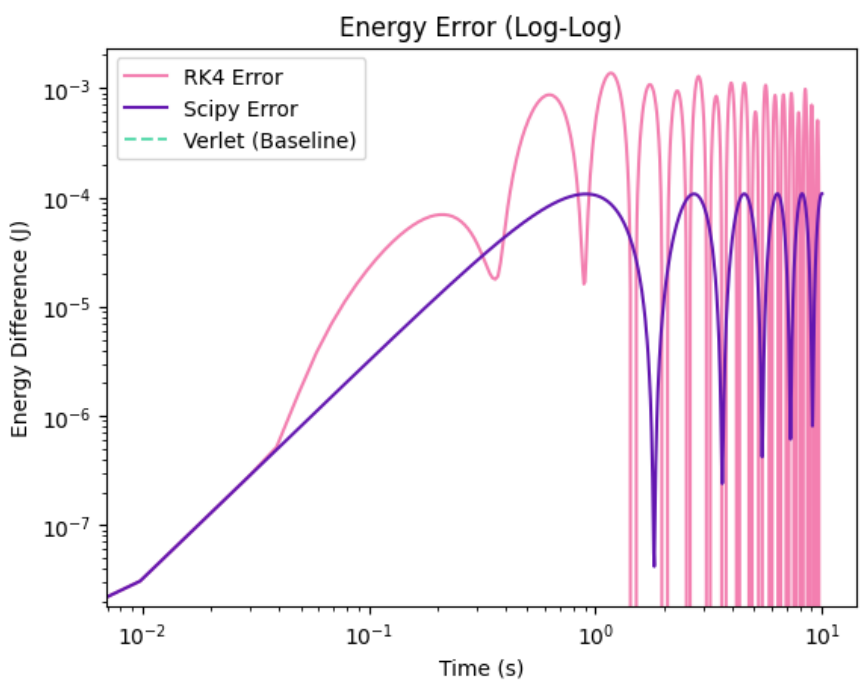

Figure 6: The error of the Scipy and RK45 integrators as compared to the Verlet integrator in calculating the total mechanical energy versus time of an undamped SHO.

The plots were generated again for the damped SHO with damping coefficient of 0.2.

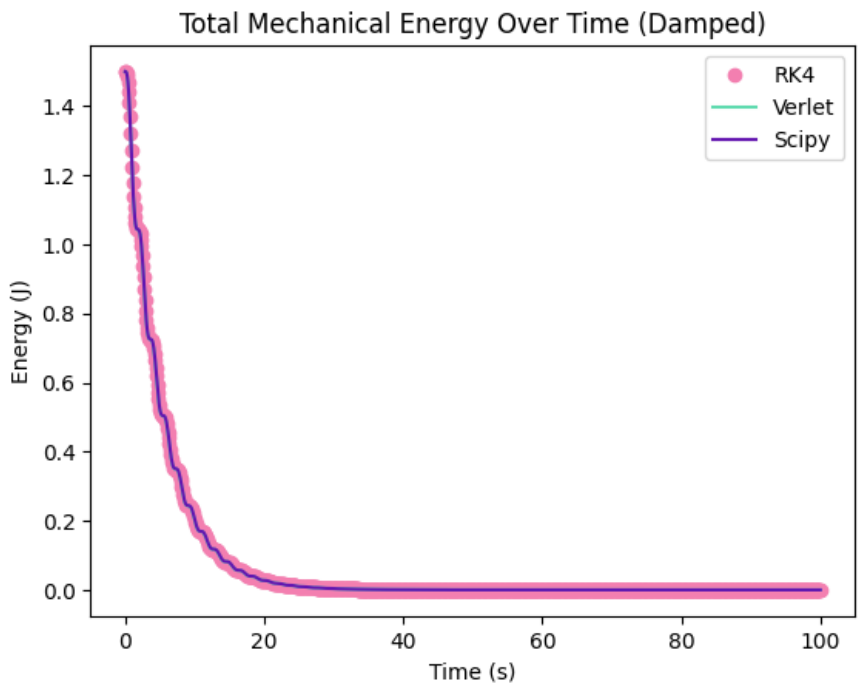

Figure 7: The total mechanical energy versus time of a damped SHO as calculated by the three integrators.

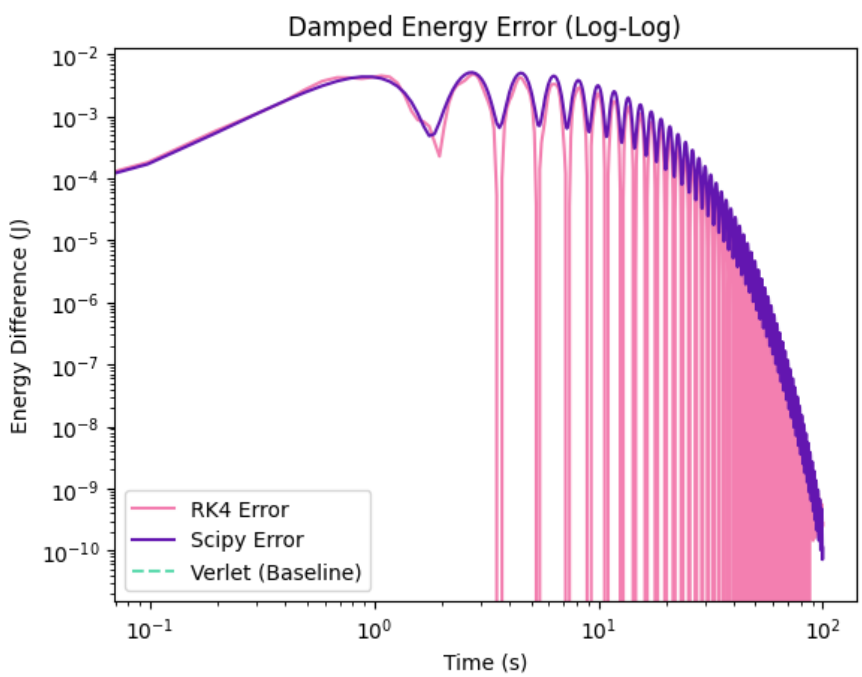

Figure 8: The error of the Scipy and RK45 integrators as compared to the Verlet integrator in calculating the total mechanical energy versus time of a damped SHO.

3.4 Extension 1

Extension 1 asks to show phase space area under time evolution. This can be done via generating a set of points, then mapping how their area evolves through time.

Listing 7: Code generating the phase space area and time arrays.

```
# set initial variables
delta = 0.1
damped = 0.2
mesh = [4, 32] # first evolving over 4 points, then 32

for Ngrid in mesh:

  x_pts = np.linspace(1 - delta, 1 + delta, Ngrid)
  v_pts = np.linspace(0 - delta, 0 + delta, Ngrid)
  pts = [(x, v) for x in x_pts for v in v_pts]

  t, straj, rtraj, vtraj, straj_d, rtraj_d, vtraj_d
    = three_solvers(pts, omega, damping)

  area_s = psa(straj)
  area_r = psa(rtraj)
  area_v = psa(vtraj)

  area_s_d = psa(straj_d)
  area_r_d = psa(rtraj_d)
  area_v_d = psa(vtraj_d)
```

There are two new function defined for this section, called "three_solvers" and "psa". Three_solvers function was made to pass in the initial points for each of the three integrators and return the damped and undamped position and velocity arrays.

Listing 8: The three_solvers function that takes in the initial points and the initial variables, then returns the damped and undamped trajectory vectors.

```
# set initial variables
# function that runs all three symplectic integrators
# for damped and undamped oscillators
def three_solvers(pts, omega, damping):
  straj, rtraj, vtraj = [], [], []
  straj_d, rtraj_d, vtraj_d = [], [], [] # damped

  for (x0,v0) in pts: # for every point in the 2D array
    st, sx, sv = scipy_method([x0, v0], tmin, tmax, nts,
        lambda N, t: deriv_odeint(N, t, omega, 0))
    straj.append((sx, sv))
```

```
    _, rx, rv = RK45_method([x0, v0], tmin, tmax, nts,
        lambda t, N: deriv_ivp(t, N, omega, 0))
    rtraj.append((rx, rv))

    _, vx, vv = verlet([x0, v0], tmin, tmax, nts,
        lambda x, v: accel(x, v, omega, 0))
    vtraj.append((vx, vv))

    _, sx_d, sv_d = scipy_method([x0, v0], tmin, tmax, nts,
        lambda N, t: deriv_odeint(N, t, omega, damped))
    straj_d.append((sx_d, sv_d))

    _, rx_d, rv_d = RK45_method([x0, v0], tmin, tmax, nts,
        lambda t, N: deriv_ivp(t, N, omega, damped))
    rtraj_d.append((rx_d, rv_d))

    _, vx_d, vv_d = verlet([x0, v0], tmin, tmax, nts,
        lambda x, v: accel(x, v, omega, damped))
    vtraj_d.append((vx_d, vv_d))

  return st, straj, rtraj, vtraj, straj_d, rtraj_d, vtraj_d
```

The second new function, "psa", calculates phase space area. This function makes use of the scipy.spatial library to use the function ConvexHull, which determines the convex hull of the points and returns an area.

Listing 9: The psa function which takes in the 2D trajectory array and gives back the area.

```
# calculates phase space area
def psa(traj_list): # psa = phase space area
  nts = len(traj_list[0][0]) # length of the 2D array
  area = np.zeros(nts) # fill it with zeroes

  for i in range(nts): # loop through each time step
    pts = np.array([[t[0][i], t[1][i]] for t in traj_list])
    hull = ConvexHull(pts) #calculates area
    area[i] = hull.volume

  return area
```

To understand the basics of the calculation, the associated plots of phase space versus time and the log-log error were shown for a four by four set of points, then for a thirty-two by thirty-two for a larger mesh grid. This was done for the damped and undamped SHO.

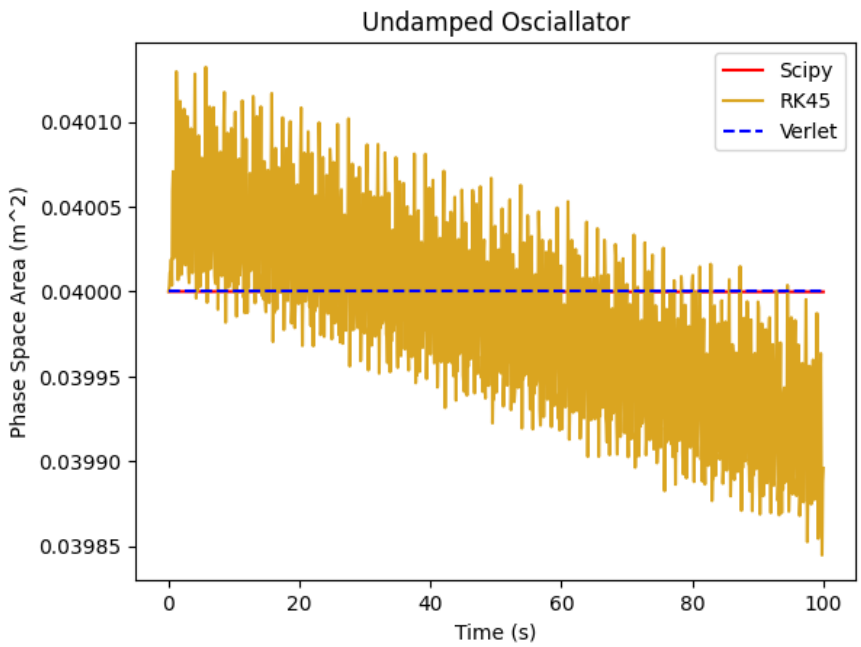

Figure 9: The evolution of the phase space area versus time for the undamped oscillator for four points.

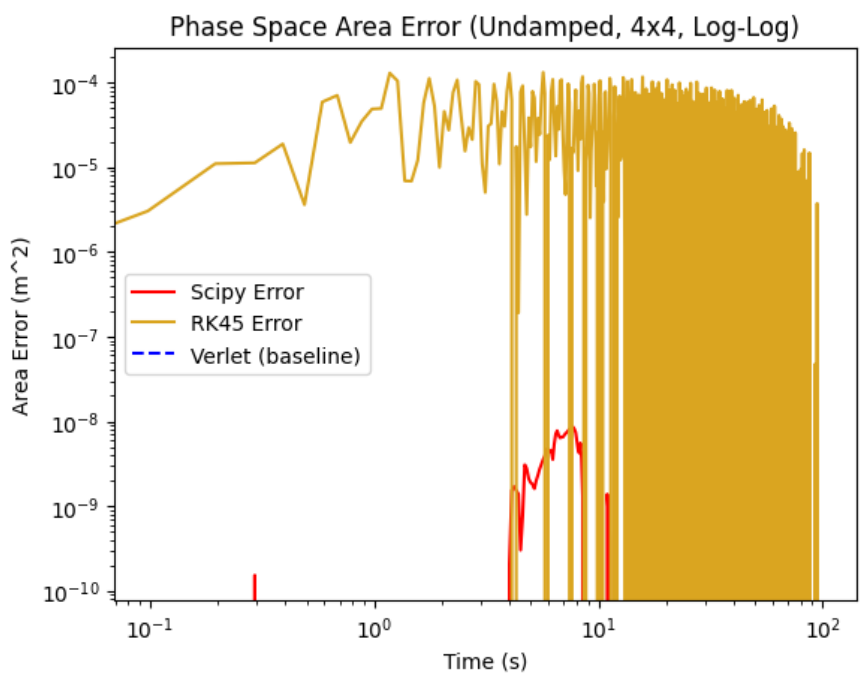

Figure 10: The log-log error evolution of the phase space area versus time for the undamped oscillator for four points.

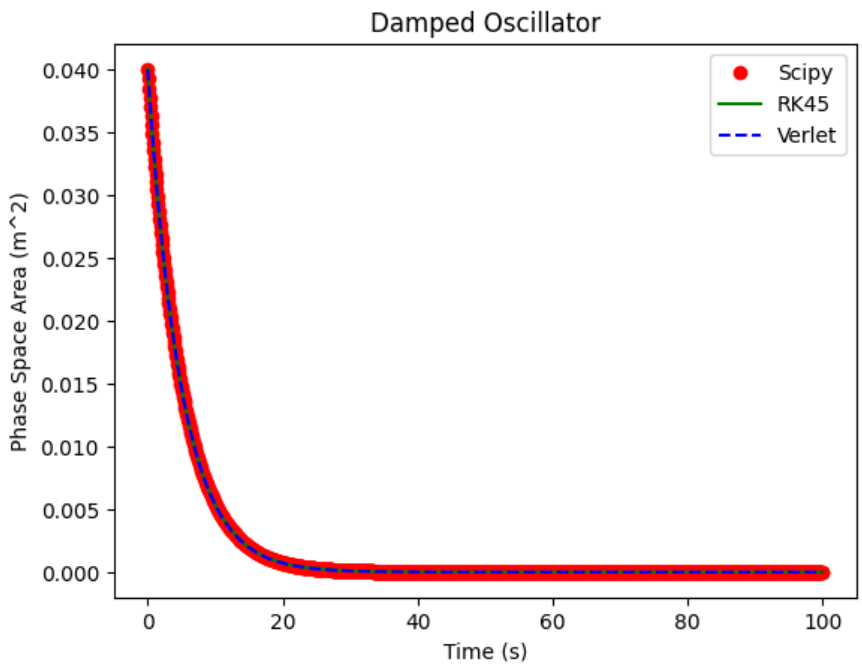

Figure 11: The evolution of the phase space area versus time for the damped oscillator for four points.

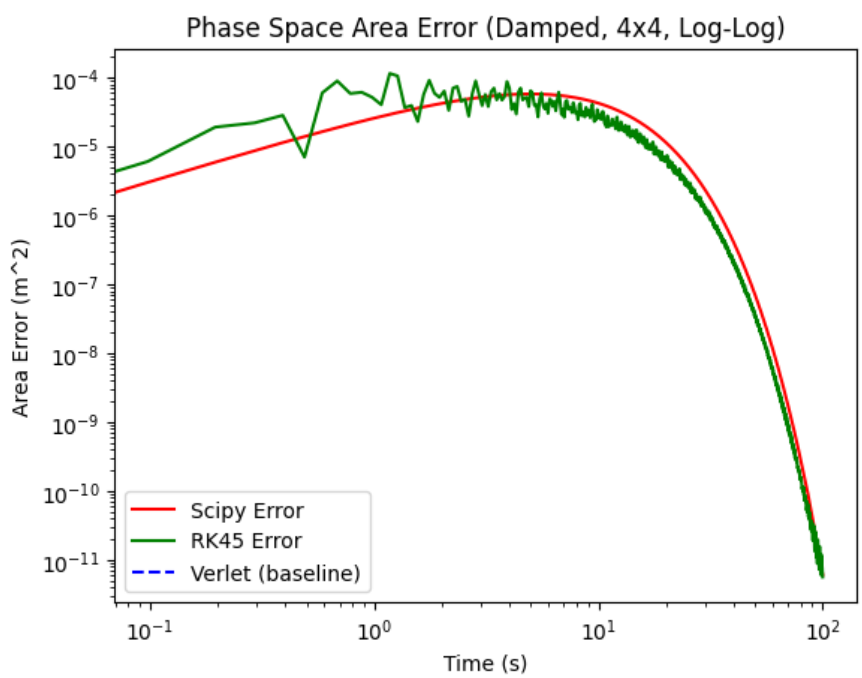

Figure 12: The log-log error evolution of the phase space area versus time for the damped oscillator for four points.

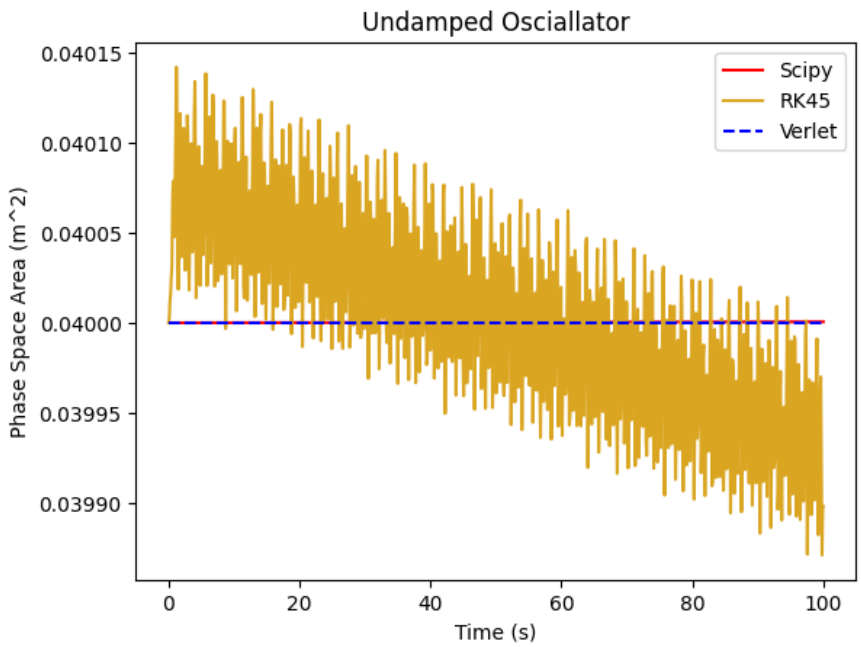

Figure 13: The evolution of the phase space area versus time for the undamped oscillator for a thirty-two point mesh grid.

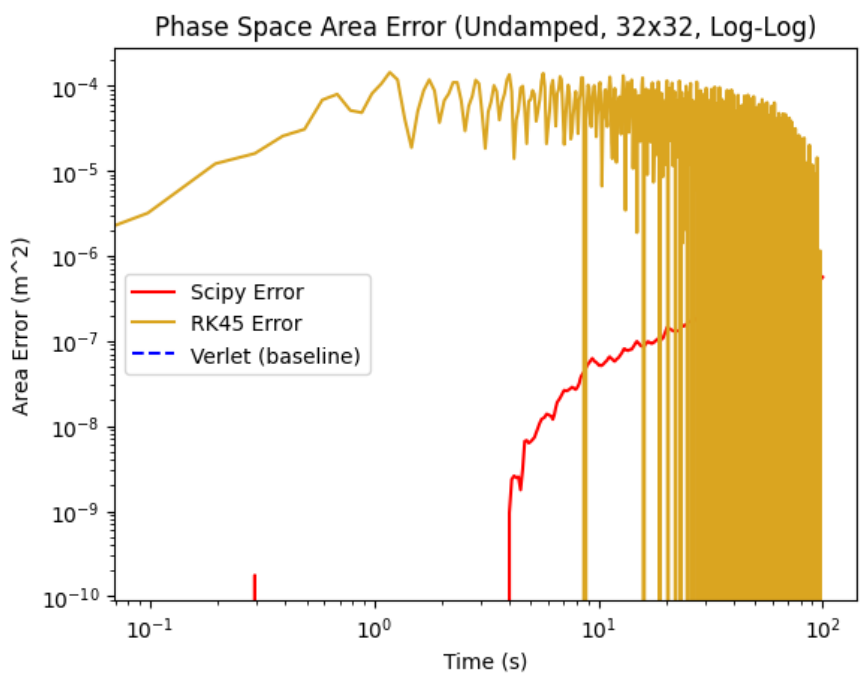

Figure 14: The log-log error evolution of the phase space area versus time for the undamped oscillator for a thirty-two point mesh grid.

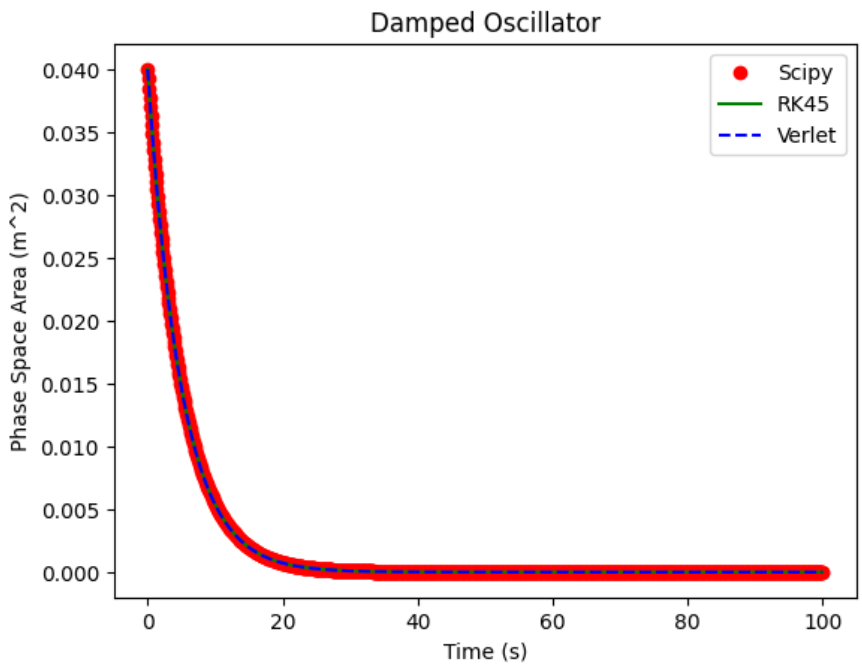

Figure 15: The evolution of the phase space area versus time for the damped oscillator for a thirty-two point mesh grid.

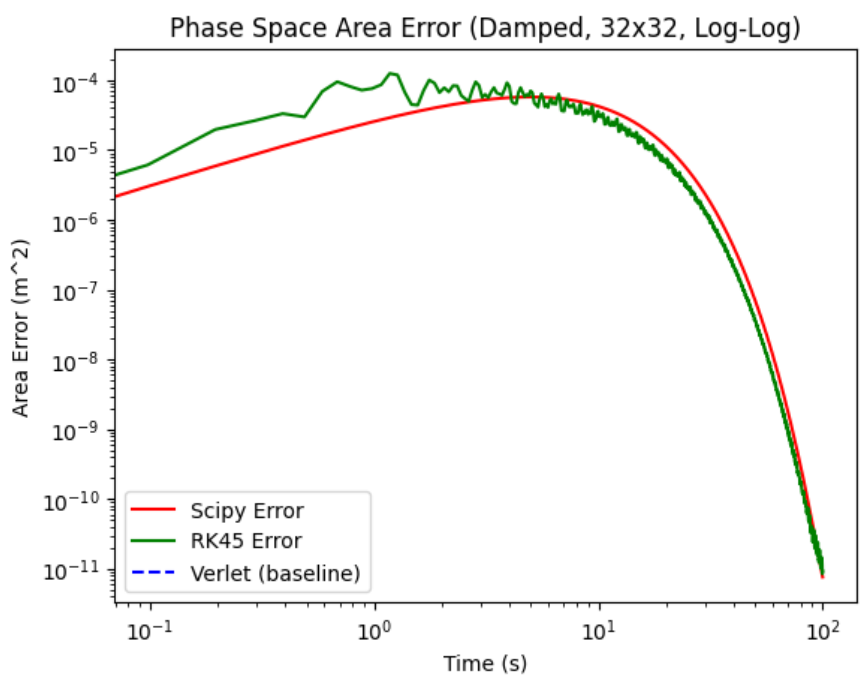

Figure 16: The log-log error evolution of the phase space area versus time for the damped oscillator for a thirty-two point mesh grid.

3.5 Extension 2

Extension 2 asked to demonstrate how the Verlet integrator conserves phase space over time. This is demonstrated by calculating the determinant of the 2D array (a matrix) and showing it is equivalent to one. This section introduces four new functions.

The first is called "onestep_verlet" in which it goes through the mathematical steps of velocity-verlet, but one for one time step instead of an array of steps. Similar code was produced for a "onestep_odeint" and a "onestep_RK45" function.

Listing 10: The onestep_verlet function. It works similar to the verlet integrator function, but only does the approximation for one time step.

```
# velocity-verlet function but only for one time step
def onestep_verlet(q, p, dt, omega, damping):
    a = -omega**2 * q - damping * p
    p_half = p + 0.5 * dt * a
    nq = q + dt * p_half
    na = -omega**2 * nq - damping * p_half
    np = p_half + 0.5 * dt * na

    return nq, np
```

The second function is called "jacobian." The code passes in the onestep_verlet function, then it uses the approximation to calculate the Jacobian matrix between the first and next value in the position and velocity arrays.

Listing 11: The jacobian function, which calculates the Jacobian 2D matrix using the onestep_verlet function.

```
# calculates jacobian for the matrix
def jacobian(int_func, qn, pn, dt, epsilon=1e-6):
  q_next, p_next = int_func(qn, pn, dt) # get result for the timestep

  qq_next, pq_next = int_func(qn + epsilon, pn, dt)
    # get result for the nudged q
  dq = (np.array([qq_next, pq_next]) - np.array([q_next, p_next]))
    / epsilon # derivative of q

  qp_next, pp_next = int_func(qn, pn + epsilon, dt)
    # get result for the nudged p
  dp = (np.array([qp_next, pp_next]) - np.array([q_next, p_next]))
    / epsilon # derivative of p

  jacobian = np.column_stack((dq, dp)) # assemble the matrix

  return jacobian
```

Now that the Jacobian can be found, the determinant of the relevant matrix is found using a handy numpy function, and the resulting determinant should show that the phase

space area is conserved over time for the undamped SHO.

Listing 12: Code calling on the jacobian function and using numpy.linalg.det() to find the determinant to see if phase space area is conserved over time for a Verlet integrator on an undamped system.

```
qn, pn = 1.0, 2.0
dt = 0.1

j = jacobian(lambda q, p, dt: onestep_verlet(q, p, dt, omega, damping),
    qn, pn, dt)
det = np.linalg.det(j)

print(f"Jacobian Matrix:\n{j}")
print("")
print(f"Determinant: {det:.1f}")
```

4 Lessons Learned

The biggest lesson from this project had to be code formatting. The simplest way to write all these functions was to make at minimum six arrays–three for each integrator, and two sets of each integrator for damped and undamped. Combine the multitude of arrays with error calculations, calling different functions, and plotting, and the code ballooned into something nearly unreadable. I spent a lot of time learning how to condense the code into a series of functions. This required me to think about how to pass in inputs that would work for both undamped and damped situation, and how that would work in execution. While the final product is still very long, I think it looks a lot better than it did. While formatting is extremely time consuming, it helps me understand what is going on better, and I think makes the code more readable to other people who may end up looking at it.

4.1 Time Keeping

I would say the code took around 10 hours to write, but that includes research and all the formatting that took place. The writeup took around six hours. This does not include the time spent in class learning about the project, discussions of the assignment among peers, or asking questions. I would say a good estimate is around twenty hours.

4.2 Attributions

- `https://www.geeksforgeeks.org/dsa/runge-kutta-4th-order-method-solve-differential-e`
  This helped me learn more about RK4 and why it is an efficient method for integration. It also had a nice explanation of how the error scales.

- `https://stackoverflow.com/questions/29009771/verlet-algorithm-implementation-in-pyt`
  This was a nice forum on the verlet method of integration. It talked a bit about coding and how the method works.

- https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.odeint.html A nice insight into the scipy.integrate odeint library/function.
- https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html#scipy.integrate.solve_ivp A nice insight into the scipy.integrate solve-ivp library/function.
- https://en.wikipedia.org/wiki/Verlet_integration#Velocity_Verlet Also a wonderful explanation into how Verlet-velocity works.
- https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.ConvexHull.html Documentation on the ConvexHull function in the scipy.spatial library.
- https://en.wikipedia.org/wiki/RungeKuttaFehlberg_method Important information on the math and error of RK45.